JavaScript Notes: DOM Manipulation & Objects

Based on app.js / index.html practice file — Daniyal's JS study notes

---

1. DOM Manipulation

1.1 Selecting elements

Two selection methods are used together in this file:

- **document.getElementById("con")** — grabs a single element by its unique id attribute. Returns one element (or null if not found).
- **div.getElementsByTagName("a")** — called on that element, returns a *live HTMLCollection* of every <a> tag inside it.

From app.js:

```
function change(){
    var div = document.getElementById("con")
    var a = div.getElementsByTagName("a")

    for (var i = 0; i<a.length; i++ ){
        a[i].innerHTML="hello"
    }
}
```

Key point — scoping the search: because getElementsByTagName is called on *div* (not on *document*), only the <a> tags inside #con are matched. The two <a> tags outside the div ("hhhhhhh" and "fffffff") are left untouched.

1.2 Looping over an HTMLCollection

a.length gives the count of matched elements, and a[i] indexes into the collection just like an array. The loop **a[i].innerHTML = "hello"** replaces the inner text/markup of every matched link with the word "hello".

innerHTML vs textContent: innerHTML parses its string as HTML (so tags inside it would render), while textContent always inserts plain text. Since the value here is a plain word, either would work — but innerHTML is the one to reach for if you ever need to insert markup, e.g. a[i].innerHTML = "Home".

1.3 Wiring it up with an event

In index.html the function is triggered inline: **<button onclick="change()">click</button>**. This is the simplest way to connect an event to a function, though in larger projects addEventListener("click", change) is generally preferred since it keeps JS out of the HTML and allows multiple listeners on one element.

1.4 Page structure recap

```
<a href="">hhhhhhh</a>          <!-- outside #con, NOT changed -->
<div id="con">
  <a href="">Home</a>        <!-- inside #con, changed to "hello" -->
```

```
  <a href="">Contact</a>     <!-- inside #con, changed to "hello" -->
  <a href="">About</a>       <!-- inside #con, changed to "hello" -->
  <button onclick="change()">click</button>
</div>
<a href="">fffffff</a>         <!-- outside #con, NOT changed -->
```

2. Working with Objects — CRUD Operations

The commented-out lines in app.js sketch the four basic operations you can perform on an object's properties (Create, Read, Update, Delete) using a hypothetical object std_1:

Operation	Syntax	What it does
Create / Insert	`std_1.address = "karachi"`	Adds a new property (address) to the object if it doesn't already exist.
Update	`std_1.id = 90`	Overwrites the existing value of a property (id) with a new one.
Delete	`delete std_1.name`	Removes a property entirely from the object.
Read	`console.log(std_1)`	Prints the object's current state to the console to inspect it.

Note: "delete" only removes properties from an object — it does not work on array elements the way you might expect (it leaves a hole/undefined rather than shrinking the array). Use array methods like splice() to actually remove array items.

3. Array of Objects + Loop

app.js also declares an array where each element is itself an object — a very common pattern for representing a list of records (here, students):

```
var students = [
    {name:'kaif', id:34, address:'karachi'},
    {name:'basit', id:35, address:'lahore'}
];

for (var i = 0; i < students.length; i++) {
    console.log(students[i].name, students[i].id);
}
```

students.length gives the number of records (2). Inside the loop, students[i] is one object (e.g. {name:'kaif', id:34, address:'karachi'}), and dot notation (students[i].name) reads a specific property from that object. This combination — array index to get the record, dot notation to get a field — is the standard way to walk through a list of structured data in JavaScript.

Output in the console would be:

```
kaif 34
basit 35
```

4. Quick Recap

- getElementById → one element by id.
- getElementsByTagName → live collection of matching tags, scoped to whatever element you call it on.
- innerHTML lets you read or overwrite an element's inner markup/text.
- Object properties can be added, updated, deleted, and read with dot notation.
- An array of objects models a list of records; loop with array indexing, then dot notation for each field.